Technical evidence: MT5 connection code, before and after

Account holder: Giovane **Date:** 2026-09-28 **Subject:** the two PC device IDs reported by the broker, and the code changes made so that all logins come from a single terminal identity

---

Introduction

This document shows the exact code that handles the MetaTrader 5 connection in my trading assistant, in its original version and in the version running today. The project is kept in a local git repository, so every "before" snippet below is a verbatim extract from the last committed revision (commit `fabbb80`), and every "after" snippet is the file as it exists right now. Nothing was rewritten for this document; the only parts omitted are sections of the application that have nothing to do with the connection.

The changes are small and concentrated in one place. There is one file that talks to MetaTrader 5, one configuration block, one line in the application startup, and one line in the Windows task installation script.

---

1. Where the MT5 connection lives

All MetaTrader 5 access goes through a single Python module, `app/core/broker.py`. It contains one class, `MT5Broker`, which is the only component in the application that ever calls the MetaQuotes Python API. No other module imports or calls MetaTrader 5 directly.

1.1 The connection class — before the change

```
"""MT5Broker — single connection point to MetaTrader 5."""
from __future__ import annotations

import logging
import time
from datetime import datetime, timedelta, timezone
from threading import Lock
from typing import Optional

import MetaTrader5 as mt5
import pandas as pd

logger = logging.getLogger(__name__)

UTC = timezone.utc

MT5_TIMEFRAMES = {
    "TIMEFRAME_M1": mt5.TIMEFRAME_M1,
    "TIMEFRAME_M5": mt5.TIMEFRAME_M5,
    "TIMEFRAME_M15": mt5.TIMEFRAME_M15,
    "TIMEFRAME_M30": mt5.TIMEFRAME_M30,
    "TIMEFRAME_H1": mt5.TIMEFRAME_H1,
    "TIMEFRAME_H4": mt5.TIMEFRAME_H4,
    "TIMEFRAME_D1": mt5.TIMEFRAME_D1,
}

# How often to run the health check (seconds)
HEALTH_CHECK_INTERVAL = 30
# Maximum number of consecutive health failures before triggering reconnect
MAX_HEALTH_FAILURES = 3
# How long to wait between reconnect attempts (seconds)
RECONNECT_BACKOFF_BASE = 5
RECONNECT_BACKOFF_MAX = 60


class MT5Broker:
    """Singleton connection to MT5 — one instance per process.

    Includes automatic health-check and self-healing reconnection.
    """

    def __init__(self, login=None, password=None, server=None):
        self._connected = False
        self._last_health_check: float = 0.0
        self._health_failures: int = 0
        self._reconnect_lock = Lock()
        self._last_error: str = ""
```

```
        self._last_successful_call: float = 0.0
        try:
            self._mt5_login: Optional[int] = int(login) if login else None
        except (TypeError, ValueError):
            self._mt5_login = None
        self._mt5_password: str = str(password or "")
        self._mt5_server: str = str(server or "")

    def initialize(self) -> bool:
        if self._connected:
            return True
        try:
            self._connected = mt5.initialize()
            if self._connected:
                self._select_symbol()
                if self._ensure_login():
                    logger.info("MT5 connected and logged in")
                    self._health_failures = 0
                    self._last_error = ""
                    self._last_successful_call = time.monotonic()
                else:
                    logger.warning("MT5 terminal connected but login failed: %s", self._last_error)
            else:
                err = mt5.last_error()
                self._last_error = str(err) if err else "unknown init error"
                logger.error("MT5 init failed: %s", self._last_error)
        except Exception:
            self._last_error = "exception during init"
            logger.exception("MT5 init exception")
            self._connected = False
        return self._connected
```

1.2 How the login is executed (this routine did not change)

For completeness, here is the routine that performs the actual account login, `_ensure_login()`. This is the only place in the application where `mt5.login()` is called. I am showing it so you can see the complete login flow; this part of the code is identical before and after the changes:

```
    def _ensure_login(self) -> bool:
        """Log the terminal into the configured account and verify it.

        Returns True when no credentials are configured (nothing to verify),
        when the terminal is already logged into the target account, or when
        mt5.login() succeeds AND account_info() confirms the login afterwards.
        """
        if not self._mt5_login:
            return True
        try:
            info = mt5.account_info()
            if info is not None and int(getattr(info, "login", 0) or 0) == self._mt5_login:
                return True
        except Exception:
            pass
        try:
            kwargs = {"login": self._mt5_login, "password": self._mt5_password}
            if self._mt5_server:
                kwargs["server"] = self._mt5_server
            if not mt5.login(**kwargs):
                err = mt5.last_error()
                self._last_error = f"mt5.login rejected: {err}"
                logger.error("MT5 login rejected for account %s: %s", self._mt5_login, self._last_error)
                return False
        except Exception:
            self._last_error = "exception during mt5.login"
            logger.exception("MT5 login exception")
            return False
        # mt5.login returned True — verify the login actually took effect
        try:
            info = mt5.account_info()
            if info is not None and int(getattr(info, "login", 0) or 0) == self._mt5_login:
                logger.info("MT5 login verified: account %s (%s)",
                            self._mt5_login, self._mt5_server or "default server")
                return True
        except Exception:
            pass
        self._last_error = f"mt5.login returned OK but account_info does not confirm account {self._mt5_
        logger.error("%s", self._last_error)
        return False
```

1.3 What the original code allowed

The important line in the original code is this one:

```
self._connected = mt5.initialize()
```

No parameters are passed. The MetaQuotes Python API has a documented default behavior in this case:

1. It attaches to the MetaTrader terminal that is already open on the machine (same installation, same data folder). Or:
2. If it cannot attach, it **starts a hidden copy of `terminal64.exe` itself** and logs into the account.

Both branches look identical from the outside; the application works either way. The risk is in branch 2: the hidden copy is a separate terminal process that opens its own login session, and that is the kind of session a broker's system can record as an additional device.

My own logs show the conditions for branch 2 were real. The Windows watchdog that keeps the system alive relaunches MT5 whenever the process is not running, and the assistant application starts right after it. There is a window in which the assistant starts while the terminal is still coming up or still closed. The watchdog log (`data/app.log`) contains these events:

```
2026-08-17 05:25:28 | MT5 terminal not running - relaunching
2026-08-17 05:25:28 | Launching MT5 via shortcut: ...\MetaTrader 5.lnk
2026-08-17 05:25:58 | MT5 started successfully (shortcut)
2026-08-31 08:06:52 | MT5 terminal not running - relaunching
2026-09-18 03:26:26 | MT5 not running - start attempt 1/3
```

And on the morning of 2026-09-21 there was a sequence of repeated application starts (roughly every 2–3 minutes between 09:44 and 10:57):

```
2026-09-21 09:44:21 | Process started (PID: 11804)
2026-09-21 09:46:50 | App exited with code
2026-09-21 09:46:55 | Process started (PID: 8820)
2026-09-21 09:49:32 | App exited with code
... (about 25 consecutive restarts in this window) ...
2026-09-21 10:57:23 | Process started (PID: 7248)
```

There was also a second trigger for branch 2. The Windows task was installed with administrator privileges (`RunLevel: Highest`), while the desktop terminal and the Jupyter notebooks run as a normal user. Windows blocks a process of one privilege level from attaching to a process of another, so in that configuration the Python API could not attach to the visible terminal and started its own copy instead.

None of this was intentional on my side. I did not write any code that opens two terminals. The behavior came from the default of the MetaQuotes API, and the changes below remove it completely.

---

2. The changes, file by file

2.1 `app/core/broker.py` — attach-only mode and a fixed terminal path

This is the core change. Two things were added to the connection class: a rule that the application may only **attach** to a terminal that is already running, and a fixed path to the installed terminal so the API always uses the same installation in standard (non-portable) mode.

Imports — before:

```
import logging
import time
```

Imports — after:

```
import logging
import os
import subprocess
import time
```

New constants and helper function — after (did not exist before):

```
# Single terminal identity policy: the app only ATTACHES to the desktop
# MetaTrader terminal (started by the watchdog). It never launches its own
# hidden terminal instance, because each terminal login session can be
# registered by the broker as a separate device.
DEFAULT_MT5_PATH = r"C:\Program Files\MetaTrader 5\terminal64.exe"
TERMINAL_PROCESS_NAME = "terminal64.exe"


def _terminal_process_running(proc_name: str = TERMINAL_PROCESS_NAME) -> bool:
    """True when the desktop MetaTrader terminal process is already running.

    Fails open (returns True) if the check itself fails, so a broken probe
    never blocks the connection.
    """
    try:
        result = subprocess.run(
            ["tasklist", "/FI", f"IMAGENAME eq {proc_name}", "/NH"],
            capture_output=True,
            text=True,
            timeout=10,
        )
        return proc_name.lower() in result.stdout.lower()
    except Exception:
        logger.exception("Terminal process check failed — assuming running")
        return True
```

This function simply asks Windows, through the `tasklist` command, whether the terminal process `terminal64.exe` is already running. That is the only place where a second terminal could come from, and now the application checks it before doing anything.

Class constructor — before:

```
    def __init__(self, login=None, password=None, server=None):
        self._connected = False
        self._last_health_check: float = 0.0
        self._health_failures: int = 0
        self._reconnect_lock = Lock()
        self._last_error: str = ""
        self._last_successful_call: float = 0.0
        try:
            self._mt5_login: Optional[int] = int(login) if login else None
        except (TypeError, ValueError):
            self._mt5_login = None
        self._mt5_password: str = str(password or "")
        self._mt5_server: str = str(server or "")
```

Class constructor — after:

```
    def __init__(self, login=None, password=None, server=None,
                 terminal_path=None, attach_only=True):
        self._connected = False
        self._last_health_check: float = 0.0
        self._health_failures: int = 0
        self._reconnect_lock = Lock()
        self._last_error: str = ""
        self._last_successful_call: float = 0.0
        self._attach_only: bool = bool(attach_only)
        self._terminal_path: Optional[str] = terminal_path or DEFAULT_MT5_PATH
        if self._terminal_path and not os.path.isfile(self._terminal_path):
            logger.warning("MT5 terminal path not found: %s — letting the "
                           "MetaTrader5 package resolve it", self._terminal_path)
            self._terminal_path = None
        try:
            self._mt5_login: Optional[int] = int(login) if login else None
        except (TypeError, ValueError):
            self._mt5_login = None
        self._mt5_password: str = str(password or "")
        self._mt5_server: str = str(server or "")
```

Connection method — before:

```
    def initialize(self) -> bool:
        if self._connected:
            return True
        try:
            self._connected = mt5.initialize()
            if self._connected:
                self._select_symbol()
                if self._ensure_login():
                    logger.info("MT5 connected and logged in")
                    self._health_failures = 0
                    self._last_error = ""
                    self._last_successful_call = time.monotonic()
                else:
                    logger.warning("MT5 terminal connected but login failed: %s", self._last_error)
            else:
                err = mt5.last_error()
                self._last_error = str(err) if err else "unknown init error"
                logger.error("MT5 init failed: %s", self._last_error)
        except Exception:
            self._last_error = "exception during init"
            logger.exception("MT5 init exception")
            self._connected = False
        return self._connected
```

Connection method — after:

```
    def initialize(self) -> bool:
        if self._connected:
            return True
        if self._attach_only and not _terminal_process_running():
            self._last_error = (
                "attach-only mode: MT5 terminal is not running — the app will "
                "NOT launch a second terminal instance; retrying automatically "
                "once the desktop terminal is open"
            )
            logger.warning("%s", self._last_error)
            return False
        try:
            kwargs: dict = {"portable": False}
            if self._terminal_path:
                kwargs["path"] = self._terminal_path
            self._connected = mt5.initialize(**kwargs)
            if self._connected:
                self._select_symbol()
                if self._ensure_login():
                    logger.info("MT5 connected and logged in")
                    self._health_failures = 0
                    self._last_error = ""
                    self._last_successful_call = time.monotonic()
                else:
                    logger.warning("MT5 terminal connected but login failed: %s", self._last_error)
            else:
                err = mt5.last_error()
                self._last_error = str(err) if err else "unknown init error"
                logger.error("MT5 init failed: %s", self._last_error)
        except Exception:
            self._last_error = "exception during init"
            logger.exception("MT5 init exception")
            self._connected = False
        return self._connected
```

Why this change works. The new guard runs before any call to the API. If the desktop terminal is not running, the method returns immediately with a clear log message and does not call `mt5.initialize()` at all — so the API never gets the chance to start a hidden copy. If the terminal is running, the call is made with `path=` pointing to the exact installed terminal and `portable=False`, which forces the API to use the same installation and the same data folder as the desktop terminal. There is no code path left that can create a terminal with a different identity.

If the terminal is closed, the application simply waits in degraded mode and retries automatically every health-check cycle until the desktop terminal is available. Trading is not attempted without the terminal.

2.2 `app/config.py` — the two new settings

Before:

```
    "MT5_SERVER": "",
    "MT5_LOGIN": "",
    "MT5_PASSWORD": "",
}
```

After:

```
    "MT5_SERVER": "",
    "MT5_LOGIN": "",
    "MT5_PASSWORD": "",
    # Single terminal identity: exact exe path + attach-only mode, so the app
    # never spawns a hidden second MT5 terminal (broker device tracking).
    "MT5_PATH": r"C:\Program Files\MetaTrader 5\terminal64.exe",
    "MT5_ATTACH_ONLY": "1",
}
```

`MT5_PATH` is the exact executable of the installed terminal, and `MT5_ATTACH_ONLY` switches the attach-only rule on. Both have safe defaults, so even a fresh configuration behaves correctly.

2.3 `app/main.py` — wiring the settings into the single broker object

Before:

```
    broker = MT5Broker(
        login=config.get_str("MT5_LOGIN") or None,
        password=config.get_str("MT5_PASSWORD") or None,
        server=config.get_str("MT5_SERVER") or None,
    )
```

After:

```
    broker = MT5Broker(
        login=config.get_str("MT5_LOGIN") or None,
        password=config.get_str("MT5_PASSWORD") or None,
        server=config.get_str("MT5_SERVER") or None,
        terminal_path=config.get_str("MT5_PATH") or None,
        attach_only=config.get_str("MT5_ATTACH_ONLY", "1") != "0",
    )
```

The broker object is created exactly once, when the application starts, and is stored on the application object:

```
    # ── Attach to app ──────────────────────────────────────
    app.broker = broker
    app.config_obj = config
    app.market = market
    app.risk = risk
    app.persistence = persistence
```

Every part of the application uses this one object. Flask is started without debug mode and without the automatic reloader, so the application runs as a single process:

```
        app.run(host="0.0.0.0", port=8000, debug=False, use_reloader=False)
```

That line matters for the singleton: the reloader would otherwise spawn a second process, and the second process would create its own broker object.

2.4 `install_startup.ps1` — the Windows task no longer runs elevated

The Windows task that starts MT5 and the assistant was registered with administrator privileges. The application must run at the same privilege level as the desktop terminal, otherwise the Python API cannot attach to it and falls back to starting its own copy.

Before:

```
$principal = New-ScheduledTaskPrincipal -UserID "$env:USERDOMAIN\$env:USERNAME" -LogonType Interactive
```

After:

```
# RunLevel Limited: the watchdog and everything it starts (MT5 + Flask app)
# run in the normal user context. Running elevated can force the MetaTrader5
# Python package to launch a SECOND terminal instance (different integrity
# level), which the broker may register as an extra device.
$principal = New-ScheduledTaskPrincipal -UserID "$env:USERDOMAIN\$env:USERNAME" -LogonType Interactive
```

The task was re-registered with this setting on 2026-09-25. The current state of the task on this machine, queried directly from Windows Task Scheduler, is:

```
UserId    : gk
LogonType : Interactive
RunLevel  : Limited
```

2.5 `notebooks/analise_direcao_diaria.ipynb` — the analysis notebook

This notebook fetches historical candles for offline analysis and also uses the API directly. It previously made the same parameterless call, so it was changed the same way.

Before:

```
if not mt5.initialize():
    raise RuntimeError(f"MT5 init falhou: {mt5.last_error()}")
```

After:

```
# Sempre usa o MESMO terminal instalado (modo padrão, nunca portátil)
# para manter uma única identidade de dispositivo na corretora.
MT5_EXE = r"C:\Program Files\MetaTrader 5\terminal64.exe"
if not mt5.initialize(path=MT5_EXE, portable=False):
    raise RuntimeError(f"MT5 init falhou: {mt5.last_error()}")
```

2.6 `tests/conftest.py` — the test suite was adapted

The automated test suite uses a fake MetaTrader module so tests never depend on a real terminal. One line was added so the attach-only check does not interfere with the fake module:

```
sys.modules["MetaTrader5"] = _MockMT5Module(fixture_data)
importlib.reload(app.core.broker)
# Attach-only guard: pretend the desktop terminal is always running so
# unit tests don't depend on a real terminal64 process.
app.core.broker._terminal_process_running = (
    lambda proc_name="terminal64.exe": True
)
```

After all changes, the full automated test suite passes: **255 tests passed** (only the optional tests that require a live broker session were deselected).

---

3. The singleton design

I was asked to make the singleton pattern clear, so here is the complete picture.

The class `MT5Broker` is declared as a singleton connection point ("Singleton connection to MT5 — one instance per process"). In practice the discipline is enforced at three levels:

1. **One object per process.** The object is created once in `create_app()` and stored as `app.broker`. All trading logic, the web routes, the health checks, and the Telegram notifications use `app.broker` — no module creates a second broker.
2. **One process.** Flask runs with `debug=False, use_reloader=False` (shown in 2.3), so there is no second Python process and therefore no second broker object.
3. **One terminal.** The attach-only rule guarantees the broker object can only connect to the terminal that is already running on the machine. It cannot create a second one.

The flow looks like this:

```
Windows Task Scheduler — TradeAssistantOOWatchdog (non-elevated)
    |
    +-- start.ps1 (watchdog loop)
            |
            +-- starts the desktop MT5 terminal (terminal64.exe)   <-- the ONLY terminal
            |
            +-- starts the assistant (python -m app.main)
                        |
                        +-- MT5Broker (single object, stored as app.broker)
                                    |
                                    +-- mt5.initialize(path=..., portable=False)
                                            |
                                            +-- ATTACHES to the running terminal

(before the change, when the API could not attach, it started its own
 hidden terminal64.exe copy — that path no longer exists)
```

There is also a lock inside the broker (`self._reconnect_lock = Lock()`) so that even the automatic reconnect logic cannot run twice at the same time within the process.

---

4. How we know the changes are effective

The change is verifiable directly on the machine, not just by reading the code.

One terminal process. Windows shows exactly one `terminal64.exe` :

```
PS> Get-Process terminal64 | Select-Object Id, Path, StartTime


Id    Path                                          StartTime
5332  C:\Program Files\MetaTrader 5\terminal64.exe  21/09/2026 10:47:42
```

One terminal installation. There is exactly one MetaTrader 5 installation on the machine and one terminal data folder, so the terminal has one identity:

```
C:\Program Files\MetaTrader 5\terminal64.exe
C:\Users\<user>\AppData\Roaming\MetaQuotes\Terminal\D0E8209F77C8CF37AD8BF550E51FF075
```

The scheduled task is not elevated anymore (shown in 2.4, `RunLevel: Limited`), so the assistant and the desktop terminal always run in the same Windows context and the attach always succeeds.

The application cannot start a hidden terminal by construction. Before connecting, the code asks Windows whether `terminal64.exe` is running. If it is not, the call to the MetaQuotes API is never made; the application logs the attach-only message and waits. If a hidden instance were ever created, it would have to appear as a second `terminal64.exe` in the process list — and the check above shows there is none.

The test suite validates the behavior. The 255 passing tests include the broker tests that cover the connection lifecycle, the login verification, and the reconnection logic against the mocked API.

The log evidence matches the explanation. The watchdog log shows the periods when MT5 was down and relaunched, which are exactly the periods in which the old code could produce the extra sessions. With the new code, those same situations produce the "attach-only mode" message and no second process.

---

5. Conclusion

The two device IDs came from login sessions of hidden terminal copies that the MetaQuotes Python API started automatically when it could not attach to the desktop terminal — not from any deliberate second instance in my application, and not from a second physical machine. The changes in this document remove that possibility at the source: the application now attaches only to the one terminal installed on this computer, at the same privilege level, with a fixed executable path and non-portable mode.

I ask that the two historical device records (`8057f72fc4` and `f96df9dcff`) be removed or reset in the account panel, since no new login can generate a device identity other than the single terminal on this machine.